T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	04/09/2026	**DATE SUBMITTED:**	04/09/2026
NAME:	QUIMING, Kadaphy James A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Preparation Task:

- Create an array of elements with random values.
- The array must contain 100 elements that are not sorted.
- Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6. Create a header file for the implementation of the different sorting algorithms. Import this header file into your main.cpp.

ILO A OUTPUT

Code

```
// Capacity of the dataset
const int maxSize = 100;

// Generates Random Values
void RNG(int dataset[], int size);

// Show the contents of the datasets
void Display(int dataset[], int size);
```

Setting the maxSize of the array

Initialization of RNG and Display functions

```
void RNG(int dataset[], int size){
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        int RNG_Limit = 100;    // Limit of RNG
        dataset[i] = rand() % RNG_Limit + 1;
    }
}
```

Random Number Generator that generates 100 values from 1 to 100

```
void Display(int dataset[], int size){
    for(int i = 0; i < maxSize; i++){
        std::cout << dataset[i] << "    ";
    }
}
```

Display Function

```
int dataset[maxSize];
RNG(dataset, maxSize);
std::cout << "Unsorted List: \n";
Display(dataset, maxSize);
```

Calling the functions

Console Screenshot

```
Unsorted List:
21   72   49   68   5   20   100   56   97   92   29   67   86   90   23   68   22   80   46   29
82   58   80   19   2   95   37   13   2   59   26   14   62   95   19   73   93   99   23   24   17
   38   88   7   78   92   91   86   100   88   93   61   13   74   23   72   84   82   27   10   51
   71   59   9   7   55   49   87   81   3   32   56   59   29   78   51   57   80   91   97   18
76   1   48   34   95   8   24   68   20   23   60   11   41   36   19   32   58   8   64
```

Observations	The program generated a dataset of 100 random numbers ranging from 1 to 100.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

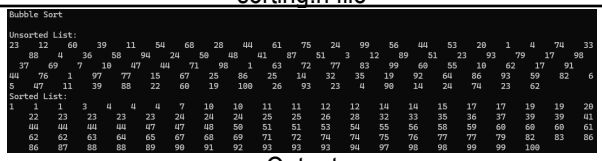
```
//BubbleSort
bubbleSort(dataset, maxSize);
std::cout << "\nSorted List: \n";
Display(dataset, maxSize);
```

main.cpp

```
// Bubble Sort
template<typename T>
void bubbleSort(T arr[], int maxSize){

    for(int i = 0; i < maxSize; i++)
        for(int j = 0; j < maxSize - i - 1; j++){
            // Compares 2 adjacent element
            if(arr[j] > arr[j + 1]){
                std::swap(arr[j], arr[j + 1]);
            }
        }
}
```

sorting.h file



Output

Observations	Bubble sort compares the adjacent values and swaps them if the first element is larger than the second element. It passes through the datasets repeatedly, until the values are arranged ascendingly.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

```
// Selection Sort
RNG(dataset, maxSize);
std::cout << "\n\nSelection Sort\n\nUnsorted List: \n";
Display(dataset, maxSize);

selectionSort(dataset, maxSize);
std::cout << "\nSorted List: \n";
Display(dataset, maxSize);
```

main.cpp

	``
	// Selection Sort
	template<typename T>
	int routineSmallest(T arr[], int currentIndex, int maxSize){
	    int pos;
	
	    T smallestElement = arr[currentIndex];
	    pos = currentIndex;
	
	    for(int j = currentIndex + 1; j < maxSize; j++){
	        if(arr[j] < smallestElement){
	            smallestElement = arr[j];
	            pos = j;
	        }
	    }
	    return pos;
	}
	
	template<typename T>
	void selectionSort(T arr[], int maxSize){
	    int pos;
	    for(int i = 0; i < maxSize; i++){
	        pos = routineSmallest(arr, i, maxSize);
	        std::swap(arr[i], arr[pos]);
	    }
	}
	``
	Sorting.h file

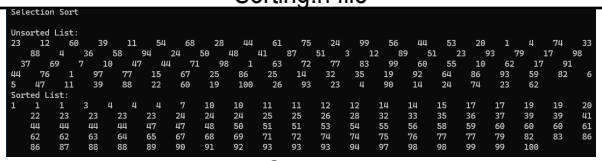
	 Output
Observations	Selection sort finds the *smallestElement* in the unsorted part of the dataset and swaps it with the first unsorted *pos.* It repeats the process until the entire dataset is sorted

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	```
	// Insertion Sort
	RNG(dataset, maxSize);
	std::cout << "\n\nInsertion Sort\n\nUnsorted List: \n";
	Display(dataset, maxSize);
	
	InsertionSort(dataset, maxSize);
	std::cout << "\nSorted List: \n";
	Display(dataset, maxSize);
	return 0;
	``
	main.cpp

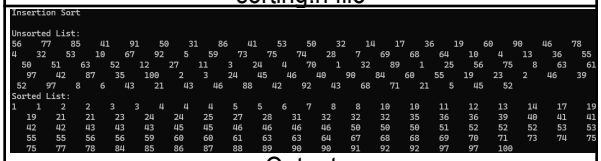
```cpp
// Insertion Sort
template<typename T>
void InsertionSort(T arr[], int maxSize){
    int j, temp;
    int k = 1;

    while(k < maxSize){
        temp = arr[k];
        j = k - 1;

        while(temp <= arr[j]){
            arr[j + 1] =  arr[j];
            j--;
        }

        arr[j + 1] = temp;
        k++;
    }
}
```

sorting.h file



Output

Observations	Insertion sort takes each value and *inserts* it into the corresponding value's position within the already sorted part of the dataset by *shifting* larger values to the right

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array **A[0…100] of unsorted elements**, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from **1 to 5**. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Step	Procedure / Process
1	START
2	SET array size to 101 **[0 to 100]**
3	GENERATE 101 random votes from 1 to 5
4	STORE the votes in the dataset (which is A[ ])
5	DISPLAY the unsorted dataset (A[ ])
6	SET votes for all candidates to 0
7	FOR each vote in the dataset: INCREMENT the vote count of the corresponding candidate
8	DISPLAY the vote count of each candidate
9	DISPLAY the winning candidate
10	END

Output Console Showing Sorted Array

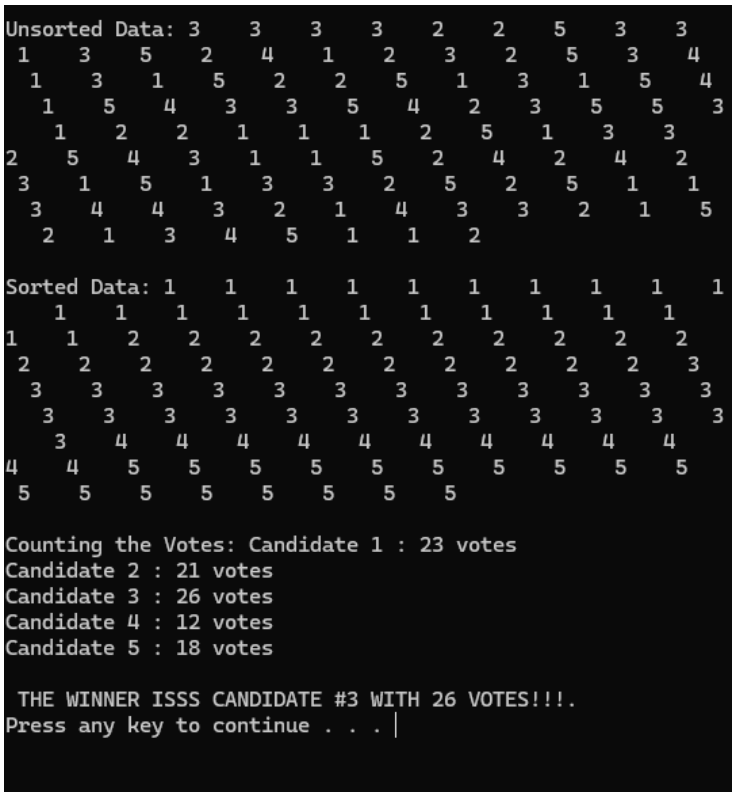
```
Sorted Data: 1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
  1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   3   3
  3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
  4   4   4   4   4   4   4   4   4   4   4   4   5   5   5   5   5   5   5   5   5   5   5   5
  5   5   5   5   5   5
```

Manual Count																1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	23	21	26	12	18
1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2					
2	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3					
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3					
3	3	3	3	3	3	4	4	4	4	4	4	4	4	4	4					
4	4	5	5	5	5	5	5	5	5	5	5	5	5	5	5					
5	5	5	5																	

Count Result of Algorithm

```
Counting the Votes: Candidate 1 : 23 votes
Candidate 2 : 21 votes
Candidate 3 : 26 votes
Candidate 4 : 12 votes
Candidate 5 : 18 votes
```

Overall Output



Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, the bubble sorting algorithm that I used was effective because it correctly counted all 100 votes and identified the candidate with the most votes.

Conclusion

I learned how to generate, sort, and count data using basic algorithms. I enjoyed the activity because it has improved my understanding of the algorithms. Overall, I did well, but I hope that I can improve my coding accuracy and knowledge on algorithms outside of the 3 that we discussed.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.